



RAFIQ VIRTUAL OFFICE

A comprehensive web-based virtual workspace platform for structured software development companies – built by the Department of Computer Systems Engineering, Helwan University, 2026.

SUPERVISED BY: DR. SAMAR ASHRAF

prepared by:

- Youssef Mahmoud Sayed - Kareem Arafat Hassaan - Youssef Farag Hassan - Youssef Wael Mahmoud

ABSTRACT

RafiQ Virtual Office is a web-based workspace engineered for three organizational roles: **Managers, Team Leaders, and Developers**. Built on Spring Boot with an H2 database and RESTful APIs, it delivers:

- Multi-stage 2FA authentication via Staff ID + OTP
- AI assistant for natural language platform control
- Real-time messaging, file sharing, and video calling
- Prayer notifications, task management, and live dashboards



USER ROLES

MANAGER

Full authority – creates/deletes rooms, assigns tasks to all roles, removes or promotes employees, views all company statistics.

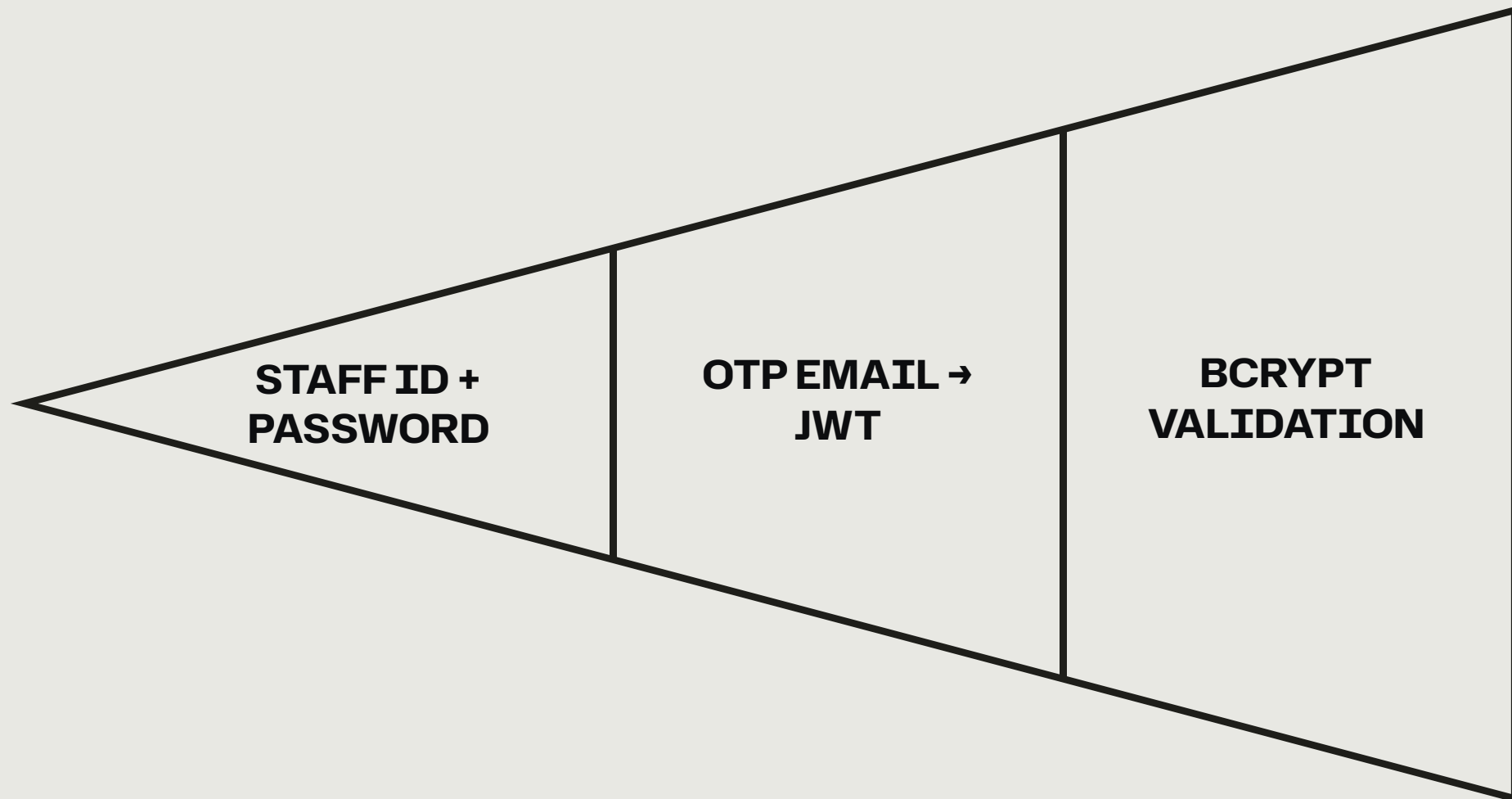
TEAM LEADER

Mid-tier – participates in rooms, assigns tasks to Developers only, receives tasks from Managers, views team-scoped dashboard.

DEVELOPER

Operational workforce – joins assigned rooms, receives tasks from Managers or Team Leaders, no assignment or room creation privileges.

AUTHENTICATION & SECURITY



Registration is Manager-initiated only – no self-signup. Passwords are BCrypt-hashed, OTPs are time-limited, and all endpoints are protected by Spring Security with `@PreAuthorize` role checks. HTTPS enforced at transport layer; all SQL queries parameterized against injection.

COMMUNICATION SYSTEM



ROOM-BASED COLLABORATION

Managers create rooms and control membership. All members communicate via:

- **Text messaging** – WebSocket / STOMP, persisted to H2
- **Voice notes** – captured via MediaRecorder API
- **File sharing** – multipart upload with download links
- **Live video calls** – WebRTC with TURN server fallback

TASK MANAGEMENT SYSTEM

1

CREATE TASK

Manager or Team Leader sets assignee, title, description, and mandatory deadline. Status initialized as **PENDING**.

2

DEADLINE ENFORCEMENT

A Spring @Scheduled job runs every minute, transitioning elapsed PENDING tasks to **OVERDUE** and locking submissions (HTTP 410).

3

COMPLETION & TRACKING

Submitted tasks move to **COMPLETED**. Dashboard updates in real time; overdue tasks are highlighted for management attention.

AI ASSISTANT & DASHBOARD

AI INTEGRATION

An external LLM (e.g., OpenAI GPT) powers a conversational assistant accessible to all roles. It parses structured intents – `CREATE_ROOM`, `ASSIGN_TASK`, `EXPLAIN_FEATURE` – and triggers the corresponding API calls automatically on the user's behalf.

LIVE DASHBOARD

Managers see total employees by role, task counts (pending / completed / overdue), and activity feeds. Team Leaders and Developers see personal task lists and room access. All stats refresh in real time via JPA aggregate queries.

APIS & TECHNOLOGIES



EMAIL API

JavaMail / Spring Mail via SMTP+TLS – Staff ID delivery, OTP, promotions, Contact Us forwarding.



PRAYER TIME API

Aladhan API fetches Fajr–Isha schedules daily. Backend broadcasts Athan audio + modal via WebSocket/SSE.



VIDEO API

WebRTC-based SDK (Jitsi / Agora) for peer-to-peer in-room video calls with TURN server fallback.



SPRING BOOT + H2

Auto-configured backend with embedded H2 (replaceable with MySQL/PostgreSQL), JPA/Hibernate ORM, Spring Security.

DATABASE DESIGN



SCHEMA HIGHLIGHTS

- Staff ID is user-facing unique identifier; integer PK used for FK relations
- Task statuses: **PENDING** → **COMPLETED** or **OVERDUE**
- All entities mapped via JPA annotations; Hibernate as ORM
- Auto-DDL on startup ensures schema consistency across deployments

PRAYER NOTIFICATION SYSTEM

The Prayer Notification System is a culturally sensitive feature designed to serve the needs of Muslim employees working within the platform. It integrates with an external Prayer Time API to deliver timely Athan notifications and prayer reminders directly within the virtual office environment, ensuring that religious obligations are not overlooked during work hours.

FIVE DAILY PRAYERS

Fajr, Dhuhr, Asr, Maghrib, and Isha schedules are retrieved from the **Aladhan Prayer Times API** using the company's geographic coordinates or city name as query parameters.

SCHEDULED FETCHING

The backend performs a single daily fetch using Spring's `@Scheduled` task. The five prayer timestamps are stored in memory or a lightweight **Caffeine Cache**, with individual timers set for each prayer event.

INCLUSIVE BY DESIGN

This feature differentiates RafiQ from generic collaboration tools that do not account for the religious practices of their users, demonstrating a genuine commitment to cultural awareness.

CONTACT US SYSTEM

The Contact Us system provides a straightforward communication channel between system users (or prospective users) and the company administration. It is accessible from a dedicated page and does **not require authentication**, making it available as a public-facing feature for external inquiries as well.

FORM FIELDS COLLECTED

- Submitter's full name
- Email address
- Subject line
- Message body

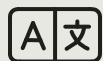
SUBMISSION & VALIDATION FLOW

The frontend sends form data as a **JSON payload** to a dedicated backend API endpoint. The backend validates all fields for completeness and format – including email address validation and message length constraints – before invoking the Email API to dispatch the inquiry to the company owner's designated email address.

The owner's email is stored as a **configuration parameter** in the Spring Boot application properties file, fully decoupled from the source code.

TECHNOLOGIES USED: BACKEND CORE

RafiQ's backend is built on a mature, enterprise-grade stack designed for reliability, security, and developer productivity.



JAVA

The primary programming language for all backend logic. Java's **strong typing, mature ecosystem, and JVM-based portability** make it an ideal choice for building reliable enterprise-grade applications.



SPRING BOOT

The backend application framework providing **auto-configuration, embedded Tomcat server, dependency injection, Spring Security, Spring Data JPA, Spring Mail, and Spring WebSocket**. Its convention-over-configuration approach significantly accelerates development.



H2 DATABASE

An **embedded, in-process relational database** engine integrating natively with Spring Boot and Hibernate. Supports full ANSI SQL, ACID transactions, and a web-based console for development-time inspection. Configurable for disk persistence or in-memory operation.

TECHNOLOGIES USED: APIS & COMMUNICATION

The platform's communication architecture relies on a set of well-established API paradigms and protocols to connect frontend clients, backend services, and external providers.



REST APIS

The system's **primary interface paradigm**. All client-server interactions are mediated through HTTP-based RESTful endpoints returning JSON-formatted responses. REST enables decoupled frontend-backend development and simplifies integration with external services.



EMAIL APIS

JavaMail / Spring Mail handles all outbound transactional emails including Staff ID delivery, OTP messages, and Contact Us forwarding via **SMTP with TLS** encryption.



VIDEO COMMUNICATION APIS

A **WebRTC-based SDK or third-party video API** (such as Jitsi or Agora) enables in-room live video calling capabilities, supporting real-time peer-to-peer communication within the virtual office.

TECHNOLOGIES USED: FRONTEND & AI

FRONTEND: HTML / CSS / JAVASCRIPT

The frontend is built with **standard web technologies**. HTML structures the content, CSS (with responsive design principles) governs the presentation, and vanilla JavaScript or a lightweight framework handles dynamic behavior, API calls, WebSocket communication, and DOM manipulation. This approach ensures broad browser compatibility and minimal dependency overhead.

AI APIS

An external **large language model API** – such as OpenAI GPT-4 or an equivalent provider – powers the AI assistant feature. It provides natural language understanding and structured intent generation, enabling intelligent, context-aware interactions within the virtual office environment.



FULL TECHNOLOGY STACK

A consolidated view of all technologies powering the RafiQ Virtual Office platform, spanning backend, frontend, data, communication, and AI layers.

Technology	Role	Key Capability
Java	Backend language	Strong typing, JVM portability
Spring Boot	Backend framework	Security, JPA, Mail, WebSocket
H2 Database	Data persistence	Embedded SQL, ACID, web console
REST APIs	Client-server interface	JSON over HTTP, decoupled design
HTML/CSS/JS	Frontend layer	Responsive UI, DOM, WebSocket
JavaMail / Spring Mail	Email dispatch	SMTP with TLS, transactional email
AI API (GPT-4)	AI assistant	NLU, intent generation
WebRTC / Jitsi / Agora	Video communication	Real-time in-room video calling
Aladhan Prayer API	Prayer scheduling	Daily prayer times by location
Caffeine Cache	In-memory caching	Lightweight prayer time storage



CONCLUSION

RafiQ Virtual Office delivers a **secure, role-enforced, feature-rich** digital workspace that generic tools cannot replicate. It demonstrates sound engineering principles: layered security, normalized schema, modular API design, and cultural inclusivity.

IMMEDIATE ROADMAP

Migrate to MySQL/PostgreSQL, develop mobile apps, implement end-to-end encryption.

CORE ACHIEVEMENT

A technically rigorous graduation project that meaningfully advances collaborative engineering tooling.

THANK YOU

Thank you for your time and attention. RafiQ Virtual Office is a reflection of thoughtful engineering, collaboration, and a commitment to building secure, inclusive digital workspaces.

We appreciate the opportunity to share this project and its vision for better teamwork.

